CPSC 3660 Database Systems
Comprehensive Final Exam Study Guide

Compiled for the 3-hour paper-based cumulative final

Final Exam: 3 hours; 120 marks; 16 questions; paper-based; cumulative.
Coverage: DB concepts, ER modeling, relational algebra, SQL, functional dependencies, normalization, brief OO-DB/NoSQL.
Budget: 1.5 minutes per mark; everything must be writable by hand.

About this guide. Built by reverse-engineering every question across Assignments 1-4 and the midterm, then generating content calibrated to the professor's exact question-construction signature. The companion HTML site adds interactive walkthroughs, KaTeX-rendered math, and a collapsible-solution practice final.
Online version: gbasran.github.io/cpsc3660-course-guide
Companions: Practice Final PDF · Practice Final Solutions PDF
The Signature: see the forensic pattern analysis for why the practice exam is shaped the way it is.

Contents

1 Part I: Database Foundations **4**
1.1 Why Databases Exist . . . 4
1.2 Schema vs Instance: the Fundamental Distinction . . . 4
1.3 DBMS Architecture . . . 5
1.4 The Three-Level Schema Architecture . . . 5
1.5 Data Models in Context . . . 5
1.6 Keys: the Vocabulary of Uniqueness . . . 6
1.7 Integrity Constraints . . . 6
1.8 Cascading Operations . . . 7

2 Part II: The Entity-Relationship (ER) Model **8**
2.1 What an ER Diagram Is . . . 8
2.2 Entities and Entity Sets . . . 8
2.3 Attributes . . . 8
2.4 Relationships and Cardinality . . . 9
2.5 Participation: Total vs Partial . . . 9
2.6 Weak Entities . . . 9
2.7 Generalization, Specialization, and ISA . . . 10
2.8 ER → Relational Mapping Rules . . . 10
2.9 Worked Example 1: Car Insurance (A3 Q4 style) . . . 11

2.10 Worked Example 2: Online Bookstore with Music CDs (A3 Q8 style, generalization) . 12
2.11 Worked Example 3: Hospital Patients, Doctors, Appointments 13

3 Part III: Relational Model and Relational Algebra 15
3.1 The Relational Model in One Paragraph 15
3.2 The Bank Database (used throughout) 15
3.3 Relational Algebra Operators 15
3.3.1 Selection σ 15
3.3.2 Projection π 15
3.3.3 Cartesian Product $\times$ 16
3.3.4 Natural Join $\bowtie$ 16
3.3.5 Theta Join $\bowtie_\theta$ 16
3.3.6 Rename ρ 16
3.3.7 Set Union $\cup$, Intersection $\cap$, Difference $-$ 16
3.3.8 Aggregation γ (or $\mathcal{G}$) 16
3.3.9 Division $\div$ 16
3.4 Division Worked Example: Library 17
3.5 RA Worked Examples (Bank schema) 17
3.6 RA Insertion via Union (Midterm Q9(c) Style) 18

4 Part IV: SQL 19
4.1 DDL: Defining the Schema 19
4.2 DML: Inserts, Updates, Deletes 19
4.3 SELECT: the Building Blocks 20
4.4 Joins 20
4.5 Aggregate Functions 21
4.6 GROUP BY and HAVING 21
4.7 Views 21
4.8 Subqueries 22
4.9 NOT EXISTS + EXCEPT: Universal Quantification (RA Division in SQL) 22
4.10 Set Operations 23
4.11 NULL and Three-Valued Logic 23
4.12 SQL Worked Examples (Bank schema) 23

5 Part V: RA $\leftrightarrow$ SQL Translation 26
5.1 Translation Walk-Throughs 26

6 Part VI: Functional Dependencies 28
6.1 Definition 28
6.2 Armstrong's Axioms (Three Core Rules) 28
6.3 Derived Rules (Useful Shortcuts) 28
6.4 Pseudo-transitivity Proof (A4 Q4 Style, Walked Through Verbatim) 28
6.5 Attribute Closure α^+ 28
6.6 Worked Closure Example 29
6.7 Uses of Closure 29
6.8 Finding All Candidate Keys for the Example 29
6.9 F^+: the Closure of an FD Set 30
6.10 More FD Worked Examples (A4 Q1-Q4 Style) 30

7 Part VII: Normalization and BCNF 31

Cumulative: DB concepts, ER, RA, SQL, FDs, BCNF

7.1 Why Normalize: Anomalies . . . 31
7.2 The Normal-Form Ladder . . . 31
7.3 BCNF Definition . . . 31
7.4 Testing for BCNF . . . 31
7.5 BCNF Decomposition Algorithm . . . 32
7.6 BCNF Worked Example (A4 Q6 Verbatim) . . . 32
7.7 Lossless Decomposition Test . . . 33
7.8 Lossless Example (Week 14) . . . 33
7.9 Lossy Example (Week 14) . . . 33
7.10 Dependency Preservation Test . . . 34
7.11 Dependency-Preserving Example (Week 14) . . . 34
7.12 Not-Dependency-Preserving Example (Week 14) . . . 34
7.13 3NF as a Fallback . . . 35

8 Part VIII: OO-DB, NoSQL, Data Lakes 36
8.1 Object-Oriented Databases . . . 36
8.2 NoSQL Families . . . 36
8.3 Data Lakes . . . 36
8.4 Comparison Table . . . 37

9 Part IX: Professor's Signature + Cheat Sheet 38
9.1 The Professor's Patterns . . . 38
9.2 Schema Recycling . . . 38
9.3 Pacing . . . 38
9.4 Cheat Sheet (compact reference) . . . 39
9.4.1 Relational Algebra . . . 39
9.4.2 Armstrong's Axioms . . . 39
9.4.3 Attribute Closure Algorithm . . . 39
9.4.4 BCNF Decomposition . . . 39
9.4.5 Lossless Test . . . 39
9.4.6 Dependency Preservation Test . . . 39
9.4.7 ER → Relational Mapping Cheat . . . 40
9.4.8 SQL Skeletons . . . 40
9.5 Last-Minute Reminders . . . 40

1 Part I: Database Foundations

1.1 Why Databases Exist

Imagine a small bakery that tracks recipes, orders, and staff on paper. As long as there are 5 orders a day, the owner manages. Now scale to 500 orders across three branches; the paper system collapses. Duplication explodes (each branch keeps its own staff list); inconsistency creeps in (a phone number is updated in one file but not another); and nothing can answer a question like "how many cinnamon rolls did Branch 2 sell last Tuesday?" without hours of manual counting. A database is the industrial kitchen equivalent: structured storage, defined recipes for retrieval, simultaneous access by multiple cooks, and safeguards so nobody contaminates the batter.

A database management system (DBMS) exists because flat files and in-memory structures do not scale when data is large, shared, long-lived, and must survive crashes. The problems a DBMS solves are:

- **Data redundancy and inconsistency**: the same fact is stored in many places and drifts out of sync.
- **Difficulty of access**: every new query requires a new hand-written program.
- **Data isolation**: data is scattered across files in varying formats.
- **Integrity problems**: constraints like "a worker must be assigned to an existing branch" are only enforced in application code.
- **Atomicity problems**: transfers of money between accounts must either fully succeed or fully fail; a crash mid-way should not leave one account debited without the other credited.
- **Concurrent access anomalies**: two users updating the same balance at the same time can lose an update.
- **Security problems**: not every user should see every table.

A DBMS consolidates storage, adds a query language, enforces integrity and transactions, controls concurrency, and manages authorization; all in one place.

1.2 Schema vs Instance: the Fundamental Distinction

A **schema** is the structure; the blueprint; the set of tables and their columns. A schema rarely changes; it is the skeleton of the database. An **instance** is the actual content at a moment in time; the rows living in each table.

A schema is the blueprint of a house (three bedrooms, two bathrooms, one kitchen). An instance is the furniture in the house right now (a blue couch in the living room, milk in the fridge, a sleeping dog). The couch can be replaced without changing the blueprint.

In programming language terms: a schema corresponds to a type definition (class `Employee` with fields name, salary); an instance corresponds to the value of a variable (the `Employee` object currently in memory).

1.3 DBMS Architecture

A DBMS is decomposed into four cooperating subsystems:

1. **Storage manager**: translates between the abstract relations the user sees and the bytes written to disk. It handles buffer management (keeping hot pages in RAM), file organization (heap, sorted, hash), and indexing (B+-trees, hash indexes) so queries do not scan the entire table.
2. **Query processor**: parses SQL, builds a relational-algebra tree, generates alternative execution plans, estimates their cost, and picks one. It is made of three pieces: a DDL interpreter that maintains the data dictionary; a DML compiler that produces execution plans; and a query evaluation engine.
3. **Transaction manager**: ensures ACID properties (atomicity, consistency, isolation, durability) via logging, locking, and recovery.
4. **Data dictionary (catalog)**: a meta-database that stores the schema itself (table definitions, column types, indexes, users). The query processor consults the catalog on every query.

1.4 The Three-Level Schema Architecture

The ANSI/SPARC three-level architecture gives a DBMS *data independence*. The same database is described at three increasingly abstract levels:

1. **Physical (internal) level**: how bytes are laid out on disk; which indexes exist; which pages are clustered together. Changing the physical level (adding an index, moving to SSD) must not require any change to queries; this is *physical data independence*.
2. **Logical (conceptual) level**: the relations, their columns, types, constraints; the schema a database designer reasons about. Changing a logical schema (adding a column) must not force rewriting view-based applications; this is *logical data independence*.
3. **View level**: what each user or application sees. A human-resources app sees a view that excludes salary; a payroll app sees it. Many views coexist on top of one logical schema.

Think of Google Maps. The physical level *is satellite tile servers in some data center. The* logical level *is "the map of Earth with streets, buildings, rivers". The* view level *is what you see on your phone depending on whether you are in driving mode, satellite mode, or transit mode. Tiles can be re-sharded without changing the map; new streets can be added to the map without changing the user interface.*

1.5 Data Models in Context

Our course is centered on the **relational model**, but the menu is larger:

Model	Structure	Example systems
Relational	tables (rows, columns)	PostgreSQL, Oracle, SQL Server
Hierarchical	tree of records	IBM IMS
Network	graph of records with links	IDMS, CODASYL
Object-oriented (OO-DB)	classes, inheritance, methods	db4o, ObjectDB
Object-relational	relational + UDTs, inheritance	PostgreSQL (partial)
Document	JSON/XML documents	MongoDB, Couchbase
Key-value	opaque value by key	Redis, DynamoDB
Column-family	sparse, wide rows	Cassandra, HBase
Graph	nodes and edges	Neo4j, ArangoDB

The relational model won commercial adoption because it has a declarative query language (SQL), a rigorous mathematical foundation (relational algebra), and constraint mechanisms that scale with data volume.

1.6 Keys: the Vocabulary of Uniqueness

Consider a relation $R(A_1, A_2, \ldots, A_n)$. A **superkey** is any subset of attributes whose values uniquely identify a tuple. A **candidate key** is a *minimal* superkey (no proper subset is a superkey). A **primary key** is one candidate key chosen by the designer to be the identifier. A **foreign key** in relation R is an attribute (or set) whose values are constrained to appear as primary key values in some other relation S.

A driver's licence is a candidate key (every driver has a unique number). A Social Insurance Number is a candidate key. The combination (full name + date of birth + birth city) is probably *a candidate key but carries risk. A superkey is anything that contains a candidate key: (SIN + favourite colour) still identifies you uniquely, but it is not minimal, so it is not a candidate key.*

Key Idea: Every candidate key is a superkey. The primary key is one candidate key that is designated as *the* identifier; it must be non-null and unique.

1.7 Integrity Constraints

Three families of constraint enforce that the database stays meaningful:

- **Domain constraints**: each attribute has a type (`INTEGER`, `VARCHAR(50)`, `DATE`); values outside that domain are rejected.
- **Entity integrity**: the primary key of every relation must be non-null. Without a key, the row is unidentifiable.
- **Referential integrity**: if $R.A$ is a foreign key referencing $S.B$, every non-null value of $R.A$ must exist in $S.B$. You cannot point to a row that does not exist.

Beyond these, a DBMS supports **semantic constraints** via `CHECK` (e.g. salary ≥ 0), `UNIQUE`, and `NOT NULL` markers.

1.8 Cascading Operations

When you delete or update a row that is referenced by a foreign key in another table, the DBMS needs a policy. Declared in `CREATE TABLE`:

- `ON DELETE CASCADE`: delete dependent rows too. Deleting a `customer` deletes all their `orders`.
- `ON DELETE SET NULL`: set dependent foreign key to `NULL`. Deleting a `manager` leaves their `department` with a null manager field.
- `ON DELETE SET DEFAULT`: set dependent FK to a default value.
- `ON DELETE NO ACTION` (default): reject the delete if dependents exist. This is the safest; it preserves referential integrity by refusing the parent operation.
- `ON DELETE RESTRICT`: identical to `NO ACTION` in most dialects; refuses with an error, evaluated at statement time.

The same options exist for `ON UPDATE`. Cascading is powerful but dangerous; a single delete can silently wipe thousands of rows.

Watch Out: `ON DELETE CASCADE` is convenient for owned relationships (an invoice and its line items) but catastrophic across loosely coupled relationships (accidentally deleting a user should not wipe their posts if those have independent audit value).

Exam Pattern: Opening five to six questions on the final are 2-mark concept questions: define superkey vs candidate key vs primary key; explain referential integrity; distinguish schema from instance; list ACID properties; compare physical and logical data independence. Treat these as freebies; give a one-sentence definition plus a two-sentence example and move on.

2 Part II: The Entity-Relationship (ER) Model

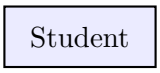
2.1 What an ER Diagram Is

An ER diagram is a pre-SQL sketch of the real world. You draw the things (entities), their attributes, and the relationships among them. Then you translate the diagram mechanically into a relational schema. The ER model is powerful because it separates two concerns: what the world looks like, and how the relational database will store it. A good ER diagram survives changes in the underlying DBMS.

An ER diagram is an architectural floor plan. The architect draws where the kitchen, bedrooms, and hallways go (entities) and how they connect (relationships). Only after the plan is agreed does anyone decide about drywall, wiring, or plumbing (the relational schema). Skipping the floor plan and going straight to drywall is how you end up with a bathroom door opening onto the stove.

2.2 Entities and Entity Sets

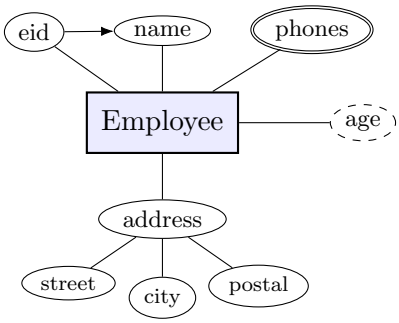
An **entity** is a distinguishable thing in the world: a specific student, a specific car, a specific invoice. An **entity set** is the collection of similar entities, usually drawn as a rectangle.



2.3 Attributes

Attributes describe an entity. Four flavours matter for the exam:

- **Simple (atomic)**: cannot be decomposed meaningfully. `age`, `salary`.
- **Composite**: has sub-parts that are themselves attributes. `address` = {street, city, postal_code}.
- **Multivalued**: a single entity has many values. `phone_numbers` for one customer. Drawn as a double-lined ellipse.
- **Derived**: computed from other attributes or from the database state. `age` from `dob`; `num_employees` from counting the employee table. Drawn as a dashed ellipse.



The primary key of an entity set is underlined in the ER diagram (here `eid`).

2.4 Relationships and Cardinality

A **relationship** connects two or more entities. Drawn as a diamond. Cardinality describes *how many* of each side participate.

- **One-to-one (1:1)**: each employee heads at most one department; each department has at most one head.
- **One-to-many (1:N)**: each department has many employees; each employee belongs to exactly one department.
- **Many-to-many (M:N)**: each student takes many courses; each course has many students.

On the diagram, the cardinality is written on the line, or encoded by 1 and N labels, or by arrows.

2.5 Participation: Total vs Partial

Participation asks whether *every* entity on one side must be in the relationship.

- **Total participation**: every employee must work for some department; drawn as a double line between entity and relationship.
- **Partial participation**: not every employee manages a project; a single line.

Here every employee works for exactly one department (total, N on this side), and a department has many employees (partial, 1 on that side; "partial" because a brand new department might have zero employees).

2.6 Weak Entities

A **weak entity** is an entity that cannot be identified by its own attributes alone. It needs the primary key of an *owner* entity plus a local *discriminator* (partial key). Drawn as a double rectangle, connected to the owner by a double-diamond identifying relationship.

A hotel room is weakly dependent on the hotel. Room 305 is not unique globally; there is a Room 305 in every hotel. To identify Room 305 uniquely you need (hotel_id, 305). Hotel_id comes from the owner; 305 is the discriminator.

In Assignment 3, a `Dependent` of an `Employee` is the canonical weak entity. A dependent's name is unique only within one employee's family; identifying a dependent globally needs (eid, dependent_name).

Key Idea: Weak entity ⇒ in the relational schema, the table has composite primary key (owner_PK, discriminator), and owner_PK is also a foreign key to the owner table with `ON DELETE CASCADE`.

2.7 Generalization, Specialization, and ISA

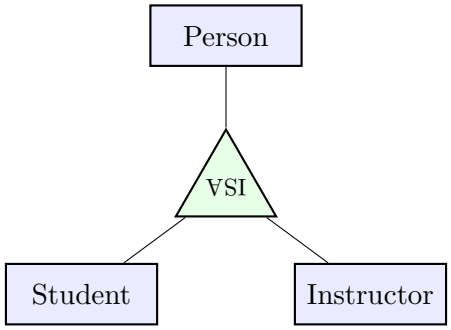
Generalization merges similar entity sets into a supertype (bottom-up). Specialization splits a supertype into subtypes (top-down). Both are drawn with an ISA triangle.

*All **Persons** have a name and SIN. A **Person** may be a **Student** (with a student id) or an **Instructor** (with a salary). Person is the supertype; Student and Instructor are subtypes. The ISA triangle reads "Student* is a *Person".*

Two orthogonal questions for every ISA:

- **Disjoint vs Overlapping**: can an entity be in two subtypes at once? Disjoint means no; a person is either a Student or an Instructor but not both. Overlapping means yes; a person could be both.
- **Total vs Partial**: must every supertype entity belong to some subtype? Total means yes; every Person must be a Student or an Instructor. Partial means a Person could be neither.

The four combinations give four ISA hierarchies: disjoint+total, disjoint+partial, overlapping+total, overlapping+partial.



2.8 ER → Relational Mapping Rules

This is the mechanical translation every exam asks for.

Rule 1: Strong entity → table.
For strong entity set E with simple attributes $a_1, \ldots, a_n$ and primary key a_1, create table $E(a_1, \ldots, a_n)$ with a_1 as primary key.

Rule 2: Weak entity → table with composite PK.
For weak entity W owned by strong entity E (PK a) with discriminator d and other attributes $b_1, \ldots, b_k$, create table $W(a, d, b_1, \ldots, b_k)$ with primary key (a, d) and foreign key (a) referencing $E(a)$ with `ON DELETE CASCADE`.

Rule 3: Multivalued attribute → separate table.
For entity E with multivalued attribute m, create table $E_m(E_PK, m)$ with primary key (E_PK, m) and foreign key $E_PK \to E(E_PK)$. An employee with multiple phone numbers becomes `Employee(eid, name)` plus `Employee_Phone(eid, phone)`.

Rule 4: Composite attribute → atomic decomposition.
Flatten. `address{street, city, postal}` becomes three columns `street, city, postal` in the entity table.

Rule 5: 1:1 relationship → merge or foreign key.
Two options. (a) Merge the two entity tables into one if participation is total on both sides. (b) Add the primary key of one side as a foreign key (with UNIQUE) in the other table. If one side has total participation, put the FK on the total side to avoid nulls.

Rule 6: 1:N relationship → foreign key on the N side.
`Department 1---N Employee`: put `did` as a foreign key in `Employee`. Relationship attributes go in the N-side table.

Rule 7: M:N relationship → junction table.
`Student M---N Course` becomes `Student(sid,...)`, `Course(cid,...)`, and a new table `Enrolls(sid, cid, grade)` where the primary key is (sid, cid) and both are foreign keys. Relationship attributes (like `grade`) live in the junction table.

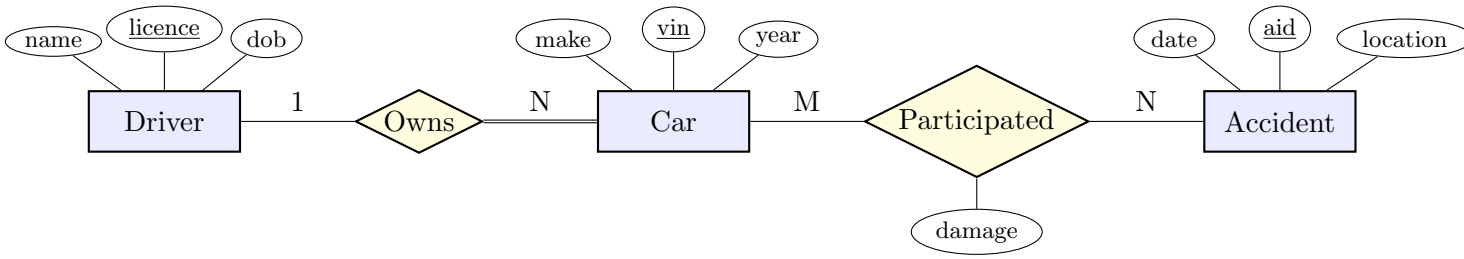
Rule 8: ISA → three strategies.

1. **Table per subtype only**: each subtype carries its own inherited columns plus its own. Good when subtypes are queried independently; bad when you need to query across the supertype.
2. **Single table with type discriminator**: one table with all supertype and subtype columns, plus a `type` column, plus nullable columns for subtype-specific attributes. Simple; wastes space.
3. **Supertype + subtype tables, shared PK**: one supertype table, one table per subtype, each with the same PK. The subtype table's PK is also a FK to the supertype. Cleanest; needs joins to reassemble.

2.9 Worked Example 1: Car Insurance (A3 Q4 style)

Requirements. A car insurance company insures drivers. Each driver has a licence number (unique), name, and birthdate. Each car has a VIN (unique), make, model, year. A driver owns zero or more cars; each car is owned by exactly one driver. The company records accidents: each accident has an accident id, date, location, and involves one or more cars. For each participating car in an accident, the damage amount is recorded.

ER diagram.



Every car is owned (total participation; double line between Owns and Car); every accident has at least one car (total); not every driver owns a car (partial); not every car is in an accident (partial).

Relational schema.

```
Driver(licence, name, dob)
Car(vin, make, model, year, owner_licence)
   FK owner_licence -> Driver(licence)
```

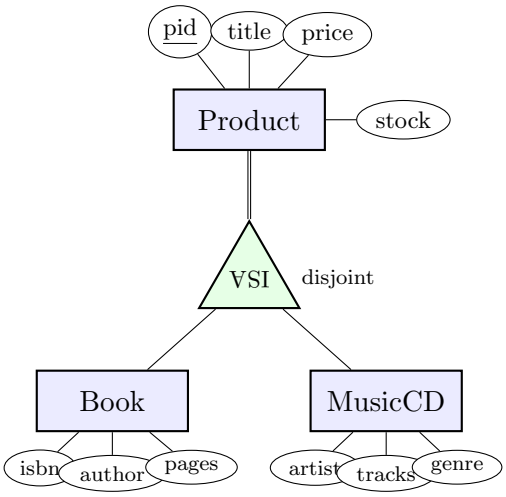
```
Accident(aid, date, location)
Participated(vin, aid, damage)
   PK (vin, aid)
   FK vin -> Car(vin)
   FK aid -> Accident(aid)
```

Owns is a 1:N relationship; it becomes a foreign key on the N side (Car). Participated is M:N; it becomes a junction table with the relationship attribute `damage`.

2.10 Worked Example 2: Online Bookstore with Music CDs (A3 Q8 style, generalization)

Requirements. An online store sells products. Every product has a product id, title, price, and stock quantity. Products are either **Books** (with an ISBN, author, page count) or **MusicCDs** (with an artist, number of tracks, genre). A product is one or the other, never both, and every product is one of the two (disjoint, total).

ER diagram.



Three mapping strategies.

Strategy 1 — table per subtype only (no supertype table):

```
Book(pid, title, price, stock, isbn, author, pages)
MusicCD(pid, title, price, stock, artist, tracks, genre)
```

Queries across all products require a `UNION`. Shared attributes are duplicated in both tables.

Strategy 2 — single table with discriminator:

```
Product(pid, title, price, stock, type,
        isbn, author, pages,      -- null when type='CD'
        artist, tracks, genre)    -- null when type='Book'
```

One table, many nulls. Simple to query the supertype; ugly for subtype constraints.

Strategy 3 — supertype + subtype (shared PK, recommended):

```
Product(pid, title, price, stock)
Book(pid, isbn, author, pages)
```

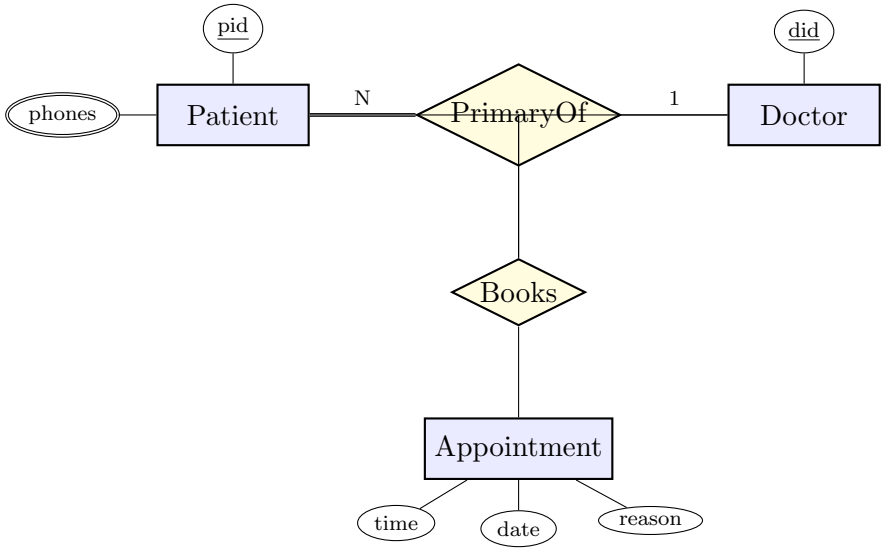
```
   FK pid -> Product(pid)
MusicCD(pid, artist, tracks, genre)
   FK pid -> Product(pid)
```

Disjoint and total can be enforced with a discriminator column plus a trigger, or with application logic.

2.11 Worked Example 3: Hospital Patients, Doctors, Appointments

Requirements. A hospital keeps patients (pid, name, dob, blood_type; plus phone numbers, of which a patient can have many). Doctors (did, name, speciality). Each patient has exactly one primary doctor; a doctor is the primary for many patients. Patients book appointments with any doctor. An appointment is identified by date, time, and doctor together; it has a reason and a status. For each appointment, there is exactly one patient and one doctor.

ER diagram.



Relational schema.

```
Patient(pid, name, dob, blood_type, primary_did)
   FK primary_did -> Doctor(did)
Patient_Phone(pid, phone)        -- multivalued
   PK (pid, phone)
Doctor(did, name, speciality)
Appointment(date, time, did, pid, reason, status)
   PK (date, time, did)
   FK did -> Doctor(did)
   FK pid -> Patient(pid)
```

Appointment is effectively weak: its identity is (date, time, did). The multivalued `phones` became its own table. The 1:N PrimaryOf became a foreign key on the N side.

Exam Pattern: ER is worth 15 to 20 marks on the final. Expect a three-part question: (a) draw the ER diagram for a described scenario (A3 Q4 style, 8 marks); (b) incorporate a multivalued attribute or weak entity (A3 Q7 style, 3 to 4 marks); (c) handle a specialization (A3 Q8 style, 4 to 6 marks) plus one of the three mapping strategies. Write the schema with FK arrows labelled; do not skip `ON DELETE CASCADE` for weak entities.

3 Part III: Relational Model and Relational Algebra

3.1 The Relational Model in One Paragraph

A **relation** is a set of tuples. A **tuple** is a finite mapping from attribute names to values. A **schema** $R(A_1, \ldots, A_n)$ fixes the attribute names and their domains. Because a relation is a set, duplicates are implicitly removed and there is no row order. NULL is a special marker meaning "value unknown or inapplicable". NULL is neither equal nor unequal to anything; comparisons involving NULL return Unknown.

3.2 The Bank Database (used throughout)

We adopt the course's running Bank schema; it appears on roughly 70% of the exam questions.

```
branch(branch_name, branch_city, assets)
customer(customer_name, customer_street, customer_city)
account(account_number, branch_name, balance)
loan(loan_number, branch_name, amount)
depositor(customer_name, account_number)
borrower(customer_name, loan_number)
```

And the Library schema for division examples:

```
employee(empid, name, department)
book(isbn, title, publisher)
borrow(empid, isbn, date)
```

3.3 Relational Algebra Operators

Relational algebra (RA) is a formal query language. It is a closed algebra: every operator takes relations and returns a relation, which lets you compose queries. Six *primitive* operators plus several derived ones:

3.3.1 Selection σ

Filter rows by a predicate. $\sigma_P(R) = \{t \in R : P(t)\}$.

> $\sigma_{\text{balance}>1000}(\text{account})$ returns all accounts with balance greater than \$1000.

Predicates combine with $\wedge, \vee, \neg$ and use $=, \neq, <, \leq, >, \geq$.

3.3.2 Projection π

Pick columns and eliminate duplicates. $\pi_{A_1,\ldots,A_k}(R)$.

> $\pi_{\text{customer_name}}(\text{depositor})$ lists each customer with an account once, no duplicates.

Watch Out: RA projection removes duplicates; SQL `SELECT` does not unless you add `DISTINCT`. This is the single most common translation error.

3.3.3 Cartesian Product $\times$

Every tuple of R paired with every tuple of S. If R has r rows and S has s rows, $R \times S$ has rs rows. Rarely useful alone; combined with selection becomes a join.

3.3.4 Natural Join $\bowtie$

Cartesian product, restricted to tuples that agree on every shared attribute, with the duplicated columns collapsed.

> depositor $\bowtie$ account pairs each depositor row with the account row that shares the same `account_number`.

3.3.5 Theta Join $\bowtie_\theta$

Cartesian product with an explicit condition: $R \bowtie_\theta S = \sigma_\theta(R \times S)$.

> employee $\bowtie_{e.\text{salary}>m.\text{salary}}$ ρ_m(employee) finds employee pairs where the first earns more than the second.

3.3.6 Rename ρ

Rename a relation or its attributes. Essential for self-joins: you cannot join a table with itself without first giving one copy a new name. $\rho_{M(A_1,\ldots,A_n)}(R)$ gives a copy called M with attributes $A_1, \ldots, A_n$.

3.3.7 Set Union $\cup$, Intersection $\cap$, Difference $-$

Require *union-compatible* relations: same number of columns, same attribute names/types in the same order.

- $R \cup S$: tuples in either.
- $R \cap S$: tuples in both.
- $R - S$: tuples in R not in S.

3.3.8 Aggregation γ (or $\mathcal{G}$)

Group and summarize. Syntax ${}_{G_1,\ldots,G_k}\mathcal{G}_{f_1(A_1),\ldots,f_m(A_m)}(R)$: group by $G_1, \ldots, G_k$, and within each group compute $f_i(A_i)$. Functions: `sum, avg, count, min, max`.

> ${}_{\text{branch_name}}\mathcal{G}_{\text{avg(balance)}}$(account) gives the average account balance per branch.

Without grouping columns, you summarise the whole relation: $\mathcal{G}_{\text{sum(amount)}}$(loan).

3.3.9 Division $\div$

The division operator answers the universal-quantifier question: "which R-tuples are paired with *every* tuple in S?"

Formally: if R has schema (X, Y) and S has schema (Y), then $R \div S$ is the set of X-values x such that for every $y \in S$, the tuple $(x, y) \in R$.

Division answers the "for all" question. Student records list which courses each student took. You want students who took every *course offered. Divide the (student, course) relation by the (course) relation. A student survives the division only if they paired with every course.*

3.4 Division Worked Example: Library

Query. "Find the names of employees who have borrowed *every* book published by Springer."

Step 1: the divisor (what every employee must match).

$$D = \pi_{\text{isbn}}(\sigma_{\text{publisher='Springer'}}(\text{book}))$$

This is a one-column relation of Springer ISBNs.

Step 2: the dividend (candidate (employee, isbn) pairs).

$$T = \pi_{\text{empid,isbn}}(\text{borrow})$$

Step 3: divide.

$$Q = T \div D$$

The result Q is a one-column relation of empid-values: those employees who appear paired with *every* Springer ISBN in T.

Step 4: get names.

$$\text{Answer} = \pi_{\text{name}}(Q \bowtie \text{employee})$$

How division is defined in terms of primitives. When your textbook asks for a division-free expression:

$$R \div S = \pi_X(R) - \pi_X\big((\pi_X(R) \times S) - R\big)$$

Read it as: start with all x-values in R; subtract those x-values for which there is *some* (x, y) expected by S that is missing from R.

3.5 RA Worked Examples (Bank schema)

Q1. Find names of customers with both an account and a loan at the Downtown branch.

$$\begin{aligned} A &= \pi_{\text{customer_name}}(\sigma_{\text{branch_name='Downtown'}}(\text{depositor} \bowtie \text{account})) \\ L &= \pi_{\text{customer_name}}(\sigma_{\text{branch_name='Downtown'}}(\text{borrower} \bowtie \text{loan})) \\ \text{Answer} &= A \cap L \end{aligned}$$

Q2. Find the branch with the highest total loan amount.

Cumulative: DB concepts, ER, RA, SQL, FDs, BCNF

Step A: total loan per branch.

$$T = {}_{\text{branch_name}}\mathcal{G}_{\text{sum(amount) as total}}(\text{loan})$$

Step B: maximum over all totals.

$$M = \mathcal{G}_{\text{max(total) as mx}}(T)$$

Step C: pick the branch(es) matching the max.

$$\text{Answer} = \pi_{\text{branch_name}}(\sigma_{\text{total=mx}}(T \times M))$$

Q3. Find customers who have an account but no loan.

$$\pi_{\text{customer_name}}(\text{depositor}) - \pi_{\text{customer_name}}(\text{borrower})$$

Q4. Self-join: customer pairs living in the same city.

$$\begin{aligned} C_1 &= \rho_{c_1}(\text{customer}) \\ C_2 &= \rho_{c_2}(\text{customer}) \\ \text{Answer} &= \pi_{c_1.\text{customer_name},c_2.\text{customer_name}}\big(\sigma_{c_1.\text{customer_city}=c_2.\text{customer_city} \wedge c_1.\text{customer_name}<c_2.\text{customer_name}}(C_1 \times C_2)\big) \end{aligned}$$

3.6 RA Insertion via Union (Midterm Q9(c) Style)

Relational algebra has no update operators per se; to *insert* a tuple, you express the new relation as the union of the old relation with the new tuple. Suppose you want to add employee (eid=5, name='Alice', salary=70000), assert she works on project p3, and make her manager of department d4:

$$\begin{aligned} \text{employee} &\leftarrow \text{employee} \cup \{(5, \text{'Alice'}, 70000)\} \\ \text{works} &\leftarrow \text{works} \cup \{(5, \text{p3})\} \\ \text{manages} &\leftarrow \text{manages} \cup \{(5, \text{d4})\} \end{aligned}$$

Each union adds one tuple if it was not already present.

Exam Pattern: RA is always 9 to 12 marks on the exam; typically three or four parts worth 2 to 3 marks each. Division and grouped aggregation are the hardest (3 marks each). Memorise the division-free expansion. Use ρ every time you self-join.

4 Part IV: SQL

SQL is the industrial language for the relational model. Every RA query has an SQL form (and a few SQL queries have no direct RA form). The course emphasises six chunks: DDL, DML, single-table `SELECT`, joins, aggregation with `GROUP BY/HAVING`, and subqueries (plus views).

4.1 DDL: Defining the Schema

```
create table branch (
  branch_name    varchar(50) primary key,
  branch_city    varchar(50) not null,
  assets         numeric(12,2) check (assets >= 0)
);

create table account (
  account_number varchar(20) primary key,
  branch_name    varchar(50) not null,
  balance        numeric(12,2) not null default 0
                  check (balance >= 0),
  foreign key (branch_name) references branch(branch_name)
    on delete no action on update cascade
);

create table depositor (
  customer_name  varchar(50),
  account_number varchar(20),
  primary key (customer_name, account_number),
  foreign key (customer_name) references customer(customer_name)
    on delete cascade,
  foreign key (account_number) references account(account_number)
    on delete cascade
);
```

Key options:

- `PRIMARY KEY`: non-null and unique.
- `UNIQUE`: like PK but allows one NULL per column.
- `NOT NULL`: disallow nulls.
- `CHECK (expr)`: enforce a boolean.
- `FOREIGN KEY (x) REFERENCES T(y) ON DELETE {CASCADE | SET NULL | NO ACTION | RESTRICT}`: referential action.

4.2 DML: Inserts, Updates, Deletes

```
insert into branch values ('Downtown', 'Lethbridge', 9000000);
insert into account (account_number, branch_name, balance)
  values ('A-101', 'Downtown', 500);

update account
set    balance = balance * 1.05
```

```
where  branch_name = 'Downtown';

delete from account where balance < 0;
```

4.3 SELECT: the Building Blocks

```
select distinct c.customer_name, c.customer_city
from   customer c
where  c.customer_city like 'Leth%'
   and c.customer_name not like '\_%'      -- '_' is a wildcard; escape with
       \
order by c.customer_name asc;
```

- `DISTINCT`: eliminate duplicates.
- `LIKE`: pattern match. `%` matches any substring; _ matches exactly one character.
- `BETWEEN lo AND hi`: inclusive range; equivalent to $\geq$ lo AND $\leq$ hi.
- `IN (v1, v2, ...)`: membership.
- `ORDER BY` supports `ASC` (default) and `DESC`.

4.4 Joins

SQL joins come in three flavours.

Implicit join via WHERE:

```
select d.customer_name, a.balance
from   depositor d, account a
where  d.account_number = a.account_number
  and  a.branch_name = 'Downtown';
```

Explicit inner join with ON:

```
select d.customer_name, a.balance
from   depositor d
join   account a on d.account_number = a.account_number
where  a.branch_name = 'Downtown';
```

Natural join (prof prefers this when the join attribute share a name):

```
select customer_name, balance
from   depositor natural join account
where  branch_name = 'Downtown';
```

Outer joins preserve unmatched rows from one or both sides:

```
select c.customer_name, a.balance
from   customer c left outer join depositor d
       on c.customer_name = d.customer_name
       left outer join account a
       on d.account_number = a.account_number;
```

A customer with no accounts still appears with NULL balance.

Watch Out: "Don't over-join." If you only need columns from one table, do not drag in a second table just because it has related data. Every needless join multiplies rows and risks duplicates.

4.5 Aggregate Functions

`COUNT(*)`, `COUNT(DISTINCT x)`, `SUM(x)`, `AVG(x)`, `MIN(x)`, `MAX(x)`. By default they operate on the whole result set.

```
select count(distinct customer_name) as num_customers_with_loans
from   borrower;
```

4.6 GROUP BY and HAVING

`GROUP BY` partitions rows by one or more columns; then aggregates apply within each group.

```
select branch_name, avg(balance) as avg_bal
from   account
group by branch_name;
```

Key Idea: Rule: every column in the `SELECT` clause must either appear in `GROUP BY` or be inside an aggregate function. Violations yield nondeterministic results and most DBs reject them.

`HAVING` filters *groups* by aggregate predicates (`WHERE` filters *rows* before grouping).

```
-- Branches whose average balance exceeds 1200:
select branch_name, avg(balance) as avg_bal
from   account
group by branch_name
having avg(balance) > 1200;
```

4.7 Views

A view is a named query; you `SELECT` from it like a table.

```
create view branch_totals as
select branch_name, sum(balance) as total_deposits
from   account
group by branch_name;
```

Views come in two flavours. A **virtual view** is recomputed on every query; it always reflects fresh data. A **materialized view** is stored and must be refreshed. SQL standard `CREATE VIEW` is virtual. Some DBs add `CREATE MATERIALIZED VIEW`.

Updating through a view is allowed only when the view is *simple* (one table, no aggregates, no `DISTINCT`, includes the primary key). Aggregated views are read-only.

4.8 Subqueries

A subquery is a `SELECT` inside another query. Four shapes matter:

IN / NOT IN

```
select customer_name from borrower
where  loan_number in (select loan_number from loan where amount > 10000);
```

EXISTS / NOT EXISTS (usually correlated)

```
select c.customer_name
from   customer c
where  exists (select 1 from depositor d where d.customer_name = c.
   customer_name);
```

ALL / ANY (SOME)

```
-- branch with the maximum asset value:
select branch_name from branch
where  assets >= all (select assets from branch);

-- loans larger than at least one loan at Downtown:
select loan_number from loan
where  amount > any (select amount from loan where branch_name = 'Downtown')
   ;
```

Correlated subquery: inner query references outer row.

```
-- customers whose every loan is under 5000:
select c.customer_name
from   customer c
where  not exists (
    select 1 from borrower b, loan l
    where  b.customer_name = c.customer_name
      and  b.loan_number = l.loan_number
      and  l.amount >= 5000);
```

4.9 NOT EXISTS + EXCEPT: Universal Quantification (RA Division in SQL)

Division translates to SQL through a double negation. "Every y in S is paired with x in R" becomes "there is no y in S not paired with x".

```
-- Employees who borrowed every book published by Springer (Library schema):
select e.name
from   employee e
where  not exists (
    (select b.isbn from book b where b.publisher = 'Springer')
    except
    (select br.isbn from borrow br where br.empid = e.empid)
);
```

Reading the pattern: the outer employee survives iff the set of "Springer ISBNs not borrowed by this employee" is empty.

Cumulative: DB concepts, ER, RA, SQL, FDs, BCNF

4.10 Set Operations

```
-- Customers with an account or a loan (or both):
(select customer_name from depositor)
union
(select customer_name from borrower);

-- Customers with both:
(select customer_name from depositor)
intersect
(select customer_name from borrower);

-- Customers with an account but no loan:
(select customer_name from depositor)
except
(select customer_name from borrower);
```

By default these remove duplicates; append `ALL` (e.g. `UNION ALL`) to keep them.

4.11 NULL and Three-Valued Logic

Any comparison with NULL returns *Unknown*. `x = NULL` is Unknown (not true, not false). Use `IS NULL` or `IS NOT NULL`.

p	q	$p \wedge q$
T	U	U
F	U	F
U	U	U

p	q	$p \vee q$
T	U	T
F	U	U
U	U	U

Aggregates ignore NULL (except `COUNT(*)`); `AVG(x)` averages only the non-null x.

4.12 SQL Worked Examples (Bank schema)

Q1. Names of all customers.

```
select distinct customer_name from depositor
union
select distinct customer_name from borrower;
```

Q2. For each branch, the number of accounts and the total balance.

```
select branch_name, count(*) as num_accts, sum(balance) as total
from   account
group by branch_name;
```

Q3. Branches whose average balance exceeds $1200.

```
select branch_name
from   account
group by branch_name
having avg(balance) > 1200;
```

Q4. Customers who have more than two accounts.

```sql
select customer_name
from   depositor
group by customer_name
having count(*) > 2;
```

Q5. Customers with an account at every branch in Lethbridge. (Division via `NOT EXISTS`/`EXCEPT`.)

```sql
select d.customer_name
from   depositor d
where  not exists (
    (select b.branch_name from branch b where b.branch_city = 'Lethbridge')
    except
    (select a.branch_name from account a
       where a.account_number = d.account_number)
)
group by d.customer_name
having count(distinct (select a.branch_name from account a
                       where a.account_number = d.account_number))
       = (select count(*) from branch where branch_city = 'Lethbridge');
```

(The tidier form joins depositor $\bowtie$ account before grouping; shown below.)

```sql
select d.customer_name
from   depositor d natural join account a
where  a.branch_name in (select branch_name from branch where branch_city='
   Lethbridge')
group by d.customer_name
having count(distinct a.branch_name)
     = (select count(*) from branch where branch_city='Lethbridge');
```

Q6. Names of customers whose loan amount exceeds the average loan amount.

```sql
select distinct b.customer_name
from   borrower b natural join loan l
where  l.amount > (select avg(amount) from loan);
```

Q7. For each customer, their total balance across all accounts.

```sql
select d.customer_name, sum(a.balance) as total_balance
from   depositor d natural join account a
group by d.customer_name
order by total_balance desc;
```

Q8. Customers whose every account is at the Downtown branch.

```sql
select d.customer_name
from   depositor d
where  not exists (
    select 1 from account a
    where  a.account_number = d.account_number
      and  a.branch_name <> 'Downtown'
);
```

Q9. Insert a new customer; give them an account at Downtown with $500.

Cumulative: DB concepts, ER, RA, SQL, FDs, BCNF

```sql
insert into customer values ('Smith', '123 Main', 'Lethbridge');
insert into account  values ('A-999', 'Downtown', 500);
insert into depositor values ('Smith', 'A-999');
```

Q10. Raise by 6% all balances at branches whose average balance is below $1000.

```sql
update account
set    balance = balance * 1.06
where  branch_name in (
    select branch_name from account
    group by branch_name
    having avg(balance) < 1000
);
```

Exam Pattern: SQL is the biggest single chunk on the final (25 to 30 marks). The sequence is: one simple `SELECT` (2m); one join (2-3m); one grouped aggregate with `HAVING` (3m); one correlated subquery (3m); one `NOT EXISTS` division (4m); one `CREATE TABLE` with FKs (3-4m); maybe one `UPDATE` or `CREATE VIEW` (2m). Write lowercase keywords cleanly; the prof reads hand-writing and abbreviations are penalised.

5 Part V: RA ↔ SQL Translation

Every RA operator has an SQL form; know both directions.

RA	SQL	Notes
$\sigma_P(R)$	`select * from R where P`	predicate directly
$\pi_{A_1,\ldots,A_k}(R)$	`select distinct` $A_1,\ldots,A_k$ `from R`	RA removes dupes; SQL needs `DISTINCT`
$R \times S$	`select * from R, S`	or `CROSS JOIN`
$R \bowtie S$	`select * from R natural join S`	match all shared attrs
$R \bowtie_\theta S$	`select * from R join S on` θ	condition join
$R \cup S$	`select ...from R union select ...from S`	schemas must match
$R \cap S$	`select ...intersect select ...`	same
$R - S$	`select ...except select ...`	same
$\rho_X(R)$	`from R as X` or `from R X`	aliasing
$\gamma_{G;f(A)}(R)$	`select G, f(A) from R group by G`	aggregate with grouping
$R \div S$	`select X from R as r where not exists ((select Y from S) except (select Y from R r2 where r2.X=r.X))`	universal quantification via double negation

5.1 Translation Walk-Throughs

Example A. RA: $\pi_{\text{customer_name}}(\sigma_{\text{amount}>5000}(\text{borrower} \bowtie \text{loan}))$.

SQL:

```
select distinct customer_name
from   borrower natural join loan
where  amount > 5000;
```

Example B. RA: ${}_{\text{branch_name}}\mathcal{G}_{\text{count(*) as n}}(\text{account})$.

SQL:

```
select branch_name, count(*) as n
from   account
group by branch_name;
```

Example C (division). RA: customers who deposit at every Lethbridge branch, expressed as

$$\pi_{\text{customer_name,branch_name}}(\text{depositor} \bowtie \text{account}) \div \pi_{\text{branch_name}}(\sigma_{\text{branch_city='Leth'}}(\text{branch})).$$

SQL:

```sql
select d.customer_name
from   depositor d
where  not exists (
    (select b.branch_name from branch b where b.branch_city = 'Lethbridge')
    except
    (select a.branch_name from account a
     where a.account_number = d.account_number)
);
```

Example D (RA from SQL). Given

```sql
select customer_name
from   depositor d, account a
where  d.account_number = a.account_number and a.balance > 1000;
```

RA:

$$\pi_{\text{customer_name}}(\sigma_{\text{balance}>1000}(\text{depositor} \bowtie \text{account})).$$

Exam Pattern: A2 Q3-Q5 established this translation pattern; expect at least one 3 to 4 mark question on the final asking for direct translation in either direction. Always write `DISTINCT` in SQL when the RA uses π. Remember ρ for self-joins.

6 Part VI: Functional Dependencies

6.1 Definition

A **functional dependency** $\alpha \to \beta$ on relation R says: for any two tuples $t_1, t_2 \in R$, if $t_1[\alpha] = t_2[\alpha]$, then $t_1[\beta] = t_2[\beta]$. In words, α's value determines β's value.

A Social Insurance Number determines a person's name: SIN → name. If two rows agree on SIN, they must agree on name. This does not say two people cannot share a name; it says no two rows with the same SIN can have different names.

A dependency is **trivial** if $\beta \subseteq \alpha$. Example: $AB \to A$ is trivial and always holds. Non-trivial dependencies carry information.

6.2 Armstrong's Axioms (Three Core Rules)

Armstrong's axioms are sound (anything derived is true) and complete (every true FD can be derived).

1. **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \to \beta$. (Trivial dependencies.)
2. **Augmentation**: if $\alpha \to \beta$, then $\alpha\gamma \to \beta\gamma$ for any γ. (You can add attributes on both sides.)
3. **Transitivity**: if $\alpha \to \beta$ and $\beta \to \gamma$, then $\alpha \to \gamma$.

6.3 Derived Rules (Useful Shortcuts)

- **Union**: if $\alpha \to \beta$ and $\alpha \to \gamma$, then $\alpha \to \beta\gamma$.
- **Decomposition**: if $\alpha \to \beta\gamma$, then $\alpha \to \beta$ and $\alpha \to \gamma$.
- **Pseudo-transitivity**: if $\alpha \to \beta$ and $\beta\gamma \to \delta$, then $\alpha\gamma \to \delta$.

6.4 Pseudo-transitivity Proof (A4 Q4 Style, Walked Through Verbatim)

Claim. If $\alpha \to \beta$ and $\beta\gamma \to \delta$, then $\alpha\gamma \to \delta$, using only Armstrong's three axioms.

1. $\alpha \to \beta$. [given]
2. $\alpha\gamma \to \beta\gamma$. [augment step 1 by γ]
3. $\beta\gamma \to \delta$. [given]
4. $\alpha\gamma \to \delta$. [transitivity of steps 2 and 3] ■

This is the archetypal four-line proof. The examiner awards partial credit even if you label a step slightly wrong, so long as the derivation is sound.

6.5 Attribute Closure α^+

Given F (a set of FDs) and attribute set α, the closure α^+ is the set of all attributes functionally determined by α under F.

Algorithm.

1. Initialise result $:= \alpha$.

2. Repeat until no change:

   For each FD $\beta \to \gamma \in F$: if $\beta \subseteq$ result, then result := result $\cup\, \gamma$.

3. Output result.

6.6 Worked Closure Example

Relation $R(A, B, C, D, E)$ with $F = \{A \to BC,\ CD \to E,\ B \to D,\ E \to A\}$. Compute A^+.

Step	FD used	Result so far
0	initialise	$\{A\}$
1	$A \to BC$ (since $A \subseteq$ result)	$\{A, B, C\}$
2	$B \to D$ (since $B \subseteq$ result)	$\{A, B, C, D\}$
3	$CD \to E$ (since $CD \subseteq$ result)	$\{A, B, C, D, E\}$
4	no new additions	$\{A, B, C, D, E\}$

Since $A^+ = R$, A is a superkey.

6.7 Uses of Closure

- **Test if α is a superkey**: compute α^+; superkey iff $\alpha^+ = R$.
- **Test if $\alpha \to \beta$ is implied by F**: compute α^+; implied iff $\beta \subseteq \alpha^+$.
- **Find candidate keys**: find all minimal α with $\alpha^+ = R$.

6.8 Finding All Candidate Keys for the Example

Continue with $F = \{A \to BC, CD \to E, B \to D, E \to A\}$.

Try single attributes:

- $A^+ = ABCDE = R$. A is a candidate key.
- $B^+ = BD$. Not a superkey.
- $C^+ = C$. No.
- $D^+ = D$. No.
- E^+: $E \to A$, then $A \to BC$, then $B \to D$, then $CD \to E$ already there. $E^+ = AEBCD = R$. Yes; E is a candidate key.

Pairs (only pairs whose closure differs from single closure are worth checking):

- BC^+: BC in, $B \to D$ adds D, $CD \to E$ adds E, $E \to A$ adds A. $BC^+ = R$. BC is a candidate key (neither B nor C alone is).
- CD^+: CD in, $CD \to E$ adds E, $E \to A$ adds A, $A \to BC$ adds B. $CD^+ = R$. CD is a candidate key.
- BD: $B \to D$, nothing new; $BD^+ = BD$. Not a key.

Candidate keys: $\{A, E, BC, CD\}$.

6.9 F^+: the Closure of an FD Set

F^+ is the set of all FDs logically implied by F. We rarely compute F^+ explicitly; we test membership of a single FD by closure. F^+ is useful conceptually: two FD sets F and G are *equivalent* iff $F^+ = G^+$.

6.10 More FD Worked Examples (A4 Q1-Q4 Style)

Q. Is $AB \to D$ implied by $F = \{A \to BC, CE \to D, B \to E\}$ on $R(A, B, C, D, E)$?

Compute $(AB)^+$:

1. Init $\{A, B\}$.
2. $A \to BC$: add C. $\{A, B, C\}$.
3. $B \to E$: add E. $\{A, B, C, E\}$.
4. $CE \to D$: add D. $\{A, B, C, D, E\}$.

Since $D \in (AB)^+$, yes, $AB \to D$ is implied.

Q. Prove the Union rule from Armstrong's axioms.
Claim: $\alpha \to \beta$ and $\alpha \to \gamma$ imply $\alpha \to \beta\gamma$.

1. $\alpha \to \beta$. [given]
2. $\alpha \to \alpha\beta$. [augment step 1 by α; so $\alpha\alpha \to \alpha\beta$, i.e. $\alpha \to \alpha\beta$]
3. $\alpha \to \gamma$. [given]
4. $\alpha\beta \to \beta\gamma$. [augment step 3 by β]
5. $\alpha \to \beta\gamma$. [transitivity of 2 and 4] ■

Exam Pattern: FD theory is worth 8 to 12 marks on the final. Expect: (a) compute an attribute closure; (b) list all candidate keys; (c) prove a derived rule from Armstrong's axioms. Pseudo-transitivity is high probability given A4 Q4 hit it. Write every step and cite the axiom.

7 Part VII: Normalization and BCNF

7.1 Why Normalize: Anomalies

Consider a single unnormalized table `StudentCourseInstructor(sid, sname, cid, ctitle, iid, iname)` where one row is (1, Alice, C1, DBMS, 100, Prof Jane).

- **Insertion anomaly**: you cannot record a new instructor without also assigning them to a student in a course.
- **Update anomaly**: if Prof Jane changes offices, you must update every row she appears in; miss one and the DB is inconsistent.
- **Deletion anomaly**: removing the last student from a course erases all record that the course ever existed.

Normalization decomposes such tables into smaller ones where each table represents one "theme".

7.2 The Normal-Form Ladder

Form	Requirement
1NF	All attribute values are atomic. No repeating groups; no lists-in-a-cell.
2NF	1NF and every non-key attribute fully depends on the whole key (no partial dependency on part of a composite key).
3NF	2NF and no transitive dependency: no non-key attribute depends on another non-key attribute.
BCNF	For every non-trivial FD $\alpha \to \beta$ in R, α is a superkey of R. Strictly stronger than 3NF in pathological cases.

Think of a filing cabinet. 1NF is: every drawer holds files (no drawer is a box of mixed receipts and post-it stickies). 2NF is: each file contains information about exactly one topic (a client file does not also contain store inventory). 3NF is: a fact is recorded in its home drawer, not replicated where it does not belong (the manager's phone is in the manager drawer, not also inside each staff file). BCNF is the strictest; every fact lives in the one and only drawer that determines it.

7.3 BCNF Definition

R is in BCNF iff for every non-trivial FD $\alpha \to \beta$ that holds on R, α is a superkey of R.

Equivalently: the only legal determinants are superkeys.

7.4 Testing for BCNF

For each FD $\alpha \to \beta$ in F:

- If $\beta \subseteq \alpha$, trivial, skip.
- Compute α^+. If $\alpha^+ = R$, α is a superkey, OK.
- Otherwise $\alpha \to \beta$ violates BCNF.

7.5 BCNF Decomposition Algorithm

Given R and F:

1. Compute all candidate keys of R.
2. Find any non-trivial FD $\alpha \to \beta$ where α is not a superkey.
3. Decompose R into $R_1 = \alpha \cup \beta$ and $R_2 = R - (\beta - \alpha)$ (i.e. R_2 keeps α plus every attribute not in $\beta - \alpha$).
4. Project F onto R_1 and R_2, obtaining F_1 and F_2.
5. Repeat on any R_i still violating BCNF.

Every decomposition produced this way is *lossless*. It may not be *dependency-preserving*; when forced to choose, 3NF is the fallback that always can be both lossless and dependency-preserving.

7.6 BCNF Worked Example (A4 Q6 Verbatim)

Given. $R(A, B, C, D, E)$ with $F = \{A \to BC,\ CD \to E,\ B \to D,\ E \to A\}$.

Step 1: candidate keys. From the previous part we already found them: $\{A, E, BC, CD\}$.

Step 2: check each FD.

FD	Is LHS a superkey?	Violates BCNF?
$A \to BC$	$A^+ = R$, yes	no
$CD \to E$	$CD^+ = R$, yes	no
$B \to D$	$B^+ = BD \neq R$, no	**yes**
$E \to A$	$E^+ = R$, yes	no

Step 3: decompose using $B \to D$.

- $R_1 = BD$ (the FD's closure of B under itself is just BD).
- $R_2 = R - \{D\} = ABCE$.

Step 4: project F onto each.

F_1 (on $R_1 = BD$): $B \to D$.

F_2 (on $R_2 = ABCE$): restrict each FD; drop any attribute not in $ABCE$. Use closure to recover implied FDs. Over R_2, working:

- $A \to BC$. (Survives as-is.)
- $E \to A$. (Survives.)
- $CD \to E$ becomes $C \to E$ only if we can derive it without D; let's check C^+ over full F: $C^+ = C$, no. So $CD \to E$ does not project.
- However, we can compute implied FDs on R_2: $A \to BC$, $E \to A$ hence $E \to BC$, and nothing else non-trivial on $ABCE$.

So $F_2 = \{A \to BC, E \to A\}$.

Step 5: check each fragment.

On $R_1 = BD$ with $F_1 = \{B \to D\}$: candidate key is B; $B \to D$ has LHS a superkey. BCNF.

Cumulative: DB concepts, ER, RA, SQL, FDs, BCNF

On $R_2 = ABCE$ with $F_2 = \{A \to BC, E \to A\}$: candidate keys are $\{A, E\}$ (both have closure $= ABCE$). $A \to BC$ LHS is a superkey; $E \to A$ LHS is a superkey. BCNF.

Final decomposition. $R_1(B, D)$ and $R_2(A, B, C, E)$, both in BCNF.

7.7 Lossless Decomposition Test

A decomposition of R into R_1, R_2 is **lossless** iff the natural join $R_1 \bowtie R_2$ returns exactly R (no spurious tuples, none missing).

Test. Decomposition is lossless iff $R_1 \cap R_2$ is a superkey of R_1 or of R_2 under F. Equivalently, $R_1 \cap R_2 \to R_1$ or $R_1 \cap R_2 \to R_2$ is in F^+.

7.8 Lossless Example (Week 14)

Relation. Borrow-loan $R(\text{customerID}, \text{loan_number}, \text{amount})$ with FD loan_number $\to$ amount.

Decomposition. $R_1(\text{customerID}, \text{loan_number})$ and $R_2(\text{loan_number}, \text{amount})$.

Intersection. $R_1 \cap R_2 = \{\text{loan_number}\}$. The FD loan_number $\to$ amount holds; so $R_1 \cap R_2 \to R_2$. Decomposition is lossless.

7.9 Lossy Example (Week 14)

Relation. Employee(EmployeeID, Project, Branch) with FDs EmployeeID $\to$ Branch and Project $\to$ Branch.

Decomposition. $R_1(\text{EmployeeID}, \text{Branch})$ and $R_2(\text{Project}, \text{Branch})$.

Intersection. $R_1 \cap R_2 = \{\text{Branch}\}$. Does Branch determine EmployeeID? No. Does Branch determine Project? No. Not lossless.

Concrete demonstration. Suppose the original instance is:

EmployeeID	Project	Branch
e1	p1	B1
e1	p2	B1
e2	p2	B2
e3	p1	B1
e3	p3	B1

Five tuples. Project R_1:

EmployeeID	Branch
e1	B1
e2	B2
e3	B1

Project R_2:

Project	Branch
p1	B1
p2	B1
p2	B2
p3	B1

Natural join $R_1 \bowtie R_2$ on Branch produces:

EmployeeID	Project	Branch
e1	p1	B1
e1	p2	B1
e1	p3	B1
(spurious) e2	p2	B2
e3	p1	B1
e3	p2	B1
(spurious) e3	p3	B1

Seven tuples; two are spurious. The original had 5 rows, the join has 7; information was lost and fabricated.

7.10 Dependency Preservation Test

Given F on R and decomposition $R_1, \ldots, R_n$, let F_i be F restricted to R_i (plus all implied FDs whose attributes lie entirely in R_i). Let $F' = F_1 \cup F_2 \cup \ldots \cup F_n$. Decomposition preserves dependencies iff $(F')^+ = F^+$.

Why it matters. If dependencies are not preserved, enforcing the original constraint requires a join every time you insert; that is expensive.

7.11 Dependency-Preserving Example (Week 14)

Relation. $R(A, B, C)$ with $F = \{A \to B, B \to C, C \to B\}$.

Decomposition. $R_1(A, C)$ and $R_2(B, C)$.

Project F.

- On $R_1 = AC$: $A^+ = ABC$, so $A \to C$ is in F^+ restricted to AC. $F_1 = \{A \to C\}$.
- On $R_2 = BC$: $B \to C$ and $C \to B$. $F_2 = \{B \to C, C \to B\}$.

$F' = \{A \to C, B \to C, C \to B\}$. Check $(F')^+$ contains every FD in F^+:

- $A \to B$: from $A \to C$ and $C \to B$ by transitivity, yes.
- $B \to C$: in F'.
- $C \to B$: in F'.

Preserved.

7.12 Not-Dependency-Preserving Example (Week 14)

Relation. $R(A, B, C)$ with $F = \{C \to B, AB \to C\}$.

Decomposition. $R_1(A, C)$ and $R_2(B, C)$.

Project.

- On $R_1 = AC$: nothing non-trivial. $F_1 = \emptyset$.
- On $R_2 = BC$: $C \to B$ holds. $F_2 = \{C \to B\}$.

$F' = \{C \to B\}$. But $AB \to C \notin (F')^+$, since from $C \to B$ alone you cannot derive $AB \to C$. Not preserved.

Lossless check: $R_1 \cap R_2 = \{C\}$; $C \to B$ makes R_2 superkey, so $C \to R_2$. Lossless but not dep-preserving.

7.13 3NF as a Fallback

Sometimes you cannot achieve both BCNF and dependency preservation. In that case 3NF is the compromise. 3NF requires: for every non-trivial FD $\alpha \to \beta$, at least one of:

- α is a superkey, or
- β is a subset of some candidate key (every attribute in β is a *prime* attribute).

3NF always admits a lossless *and* dependency-preserving decomposition. BCNF is stricter but cannot always do both.

Exam Pattern: Normalization is 15 to 20 marks on the final; often a single 15-mark showpiece. Expect: (a) state F and enumerate all candidate keys via closure; (b) check BCNF and identify violating FDs; (c) decompose to BCNF; (d) verify each fragment is lossless using the intersection test; (e) state whether dependencies are preserved. Show every closure computation. The week-14 lossless/lossy pair and the dep-preservation pair are directly testable.

8 Part VIII: OO-DB, NoSQL, Data Lakes

8.1 Object-Oriented Databases

An OO-DB stores objects with state and behaviour, supporting classes, inheritance, and encapsulation directly. The motivation is *complex types*: designs (CAD models, geometries, nested lists) that the relational model flattens awkwardly.

- **Object identity**: every object has an OID independent of its attribute values. Two objects can have identical state but be different objects.
- **Classes and inheritance**: schema defines class hierarchies directly; the ISA of the ER model is first-class in an OO-DB.
- **Methods**: objects respond to messages; behaviour lives alongside state.
- **ODL / OQL**: the Object Database Management Group defined ODL (schema) and OQL (queries).

The tradeoff: OO-DBs are expressive for complex types but weaker at ad-hoc querying and have much smaller ecosystems. *Object-relational* systems (like PostgreSQL with UDTs) blend the two.

8.2 NoSQL Families

NoSQL collects several non-relational storage paradigms that trade away ACID or schema rigidity for horizontal scale.

Family	Model	Examples	Best for
Key-value	opaque value indexed by key	Redis, DynamoDB	caching, session store
Document	JSON/BSON/XML document per key	MongoDB, Couchbase	content management, catalogs
Column-family	sparse wide rows; column families	Cassandra, HBase	time-series, telemetry
Graph	nodes + labelled edges	Neo4j, ArangoDB	social, knowledge graphs

Most NoSQL databases relax the C of ACID to eventually-consistent or tunable consistency. CAP theorem: you cannot simultaneously guarantee Consistency, Availability, and Partition tolerance; NoSQL typically picks AP while relational picks CP.

8.3 Data Lakes

A **data warehouse** stores cleaned, schema-on-write tables optimised for analytic queries. A **data lake** stores raw data (files, logs, JSON blobs) without upfront schema; schema is imposed only when reading.

- **Schema-on-write**: the DB validates types and constraints at insert time. Every write is fast to query later but slow to ingest if the source shape is unknown.
- **Schema-on-read**: writes are cheap; readers parse and validate. Great for exploratory analytics and heterogeneous sources; poor for transactional workloads.

8.4 Comparison Table

Aspect	Relational	OO-DB	NoSQL (document)
Data model	tables, tuples	classes, objects	JSON documents
Schema	strict, enforced	strict (class hierarchy)	flexible/none
Query language	SQL	OQL	API or JSON query
Joins	first-class	via relationships	rare; denormalise
Consistency	ACID	ACID	often eventual
Best at	structured transactions	complex typed structures	semi-structured, scale

Exam Pattern: OO-DB and NoSQL together are ∼5% of the final. If asked, give a one-paragraph definition, one example system, and one reason to pick it over relational. Do not overspend time here.

9 Part IX: Professor's Signature + Cheat Sheet

9.1 The Professor's Patterns

From assignments A1 through A4 and the midterm, a clear structure emerges:

1. **Five to six concept openers (12 marks)**: short-answer definitions (keys, constraints, schemas, ACID). Freebies.
2. **One RA multi-part (9-12 marks)**: three or four sub-queries. One will be grouped aggregation; one will be division.
3. **One SQL multi-part (25-30 marks)**: graduated difficulty: simple SELECT → join → GROUP BY/HAVING → correlated → NOT EXISTS + EXCEPT.
4. **One DDL (3-4 marks)**: write CREATE TABLE with FKs, ON DELETE, CHECK.
5. **One ER question (15-20 marks)**: draw a diagram from requirements, then map to relational.
6. **One FD/closure/key enumeration (8 marks)**.
7. **One Armstrong proof (4 marks)**: pseudo-transitivity or union rule.
8. **One BCNF decomposition (15 marks)**: candidate keys, BCNF test, decompose, lossless, dep-preservation.
9. **One NoSQL/OO-DB short answer (4-5 marks)**.

9.2 Schema Recycling

The prof recycles three schemas. Be fluent in all three.

- **Bank** (branch, customer, account, loan, depositor, borrower): 70-85% of queries.
- **Employee-Works** (employee, works, project, department): most relational-algebra drills.
- **Library** (employee, book, borrow): division and NOT EXISTS examples.

9.3 Pacing

120 marks / 180 minutes = 1.5 minutes per mark. A 3-mark SQL query deserves 4.5 minutes; a 15-mark BCNF showpiece deserves 22 minutes. If you are stuck at 5 minutes on a 2-mark concept question, skip and return.

9.4 Cheat Sheet (compact reference)

9.4.1 Relational Algebra

Op	Form	SQL equivalent
σ	$\sigma_P(R)$	`where P`
π	$\pi_{A,B}(R)$	`select distinct A, B`
$\times$	$R \times S$	`from R, S`
$\bowtie$	$R \bowtie S$	`natural join`
$\bowtie_\theta$	$R \bowtie_\theta S$	`join ...on ...`
$\cup, \cap, -$	set ops	`union / intersect / except`
ρ	$\rho_X(R)$	`R as X`
γ	${}_G\mathcal{G}_{f(A)}(R)$	`group by G / agg`
$\div$	$R \div S$	`not exists (...except ...)`

9.4.2 Armstrong's Axioms

- Reflexivity: $\beta \subseteq \alpha \Rightarrow \alpha \to \beta$.
- Augmentation: $\alpha \to \beta \Rightarrow \alpha\gamma \to \beta\gamma$.
- Transitivity: $\alpha \to \beta, \beta \to \gamma \Rightarrow \alpha \to \gamma$.

Derived: Union; Decomposition; Pseudo-transitivity ($\alpha \to \beta, \beta\gamma \to \delta \Rightarrow \alpha\gamma \to \delta$).

9.4.3 Attribute Closure Algorithm

```
result := alpha
repeat until no change:
    for each FD b -> c in F:
        if b subset result: result := result union c
return result
```

9.4.4 BCNF Decomposition

```
while R has some FD a -> b that violates BCNF (a not a superkey):
    R1 := a union b
    R2 := R minus (b minus a)
    project F onto R1 and R2
    recurse
```

9.4.5 Lossless Test

Decomposition of R into R_1, R_2 is lossless iff $R_1 \cap R_2 \to R_1$ or $R_1 \cap R_2 \to R_2$ holds in F^+.

9.4.6 Dependency Preservation Test

Let $F_i = F$ restricted to R_i; $F' = \bigcup F_i$. Preserved iff $(F')^+ = F^+$.

9.4.7 ER → Relational Mapping Cheat

ER construct	Relational translation
Strong entity	one table with all simple attrs; PK underlined
Weak entity	one table with (owner_PK, discriminator) as PK; FK on CASCADE
Simple attribute	column
Composite attribute	one column per leaf
Multivalued attribute	new table (entity_PK, value)
Derived attribute	not stored; computed from others
1:1 relationship	merge or FK (put FK on total side if any)
1:N relationship	FK on the N side
M:N relationship	junction table (PK1, PK2) + relationship attrs
ISA (disjoint/total)	supertype + subtype tables, shared PK
ISA (alternative A)	one table per subtype with full inherited schema
ISA (alternative B)	single table with type column + nullable subtype cols

9.4.8 SQL Skeletons

```sql
-- grouped aggregate with HAVING:
select group_col, agg(col) from R group by group_col having agg(col) op
   value;

-- correlated subquery:
select ... from R r where (not) exists (select 1 from S where S.x = r.x);

-- NOT EXISTS + EXCEPT (division):
select x from R r where not exists (
    (select y from S) except (select y from R r2 where r2.x = r.x)
);

-- CREATE TABLE with FK:
create table T (
    a int primary key,
    b int,
    c varchar(50) not null,
    foreign key (b) references U(b) on delete cascade
);
```

9.5 Last-Minute Reminders

- π removes duplicates; SQL `SELECT` does not. Write `DISTINCT`.
- Weak entities get composite PK plus `ON DELETE CASCADE`.
- Every candidate key has minimal superkey attributes; closure must equal R.
- BCNF decomposition is always lossless; may lose dependencies.
- 3NF admits lossless + dep-preserving decomposition always.
- Use `IS NULL` not `= NULL`.
- Self-joins need `rho` / aliasing in RA; aliasing in SQL.
- "Do not over-join" – only join what you need.

Cumulative: DB concepts, ER, RA, SQL, FDs, BCNF

- In RA "for all" queries, use division or its expansion.
- In SQL "for all", use `NOT EXISTS + EXCEPT`.

Good luck on the final.